# PyTorch框架

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

## 什么是PyTorch

PyTorch一个基于Python语言的深度学习框架，它将数据封装成张量（Tensor）来进行处理。

PyTorch提供了灵活且高效的工具，用于构建、训练和部署机器学习和深度学习模型。

PyTorch广泛应用于学术研究和工业界，特别是在计算机视觉、自然语言处理、强化学习等领域。

PyTorch

PyTorch的安装：pip install torch -i https://pypi.tuna.tsinghua.edu.cn/simple

## PyTorch发展历史

官网：https://pytorchorg.com
2016年Facebook正式发布了PyTorch的第一个版本。
2018年PyTorch发布了1.0版本，标志着其正式进入生产级应用阶段。

来源：中国信息通信研究院

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

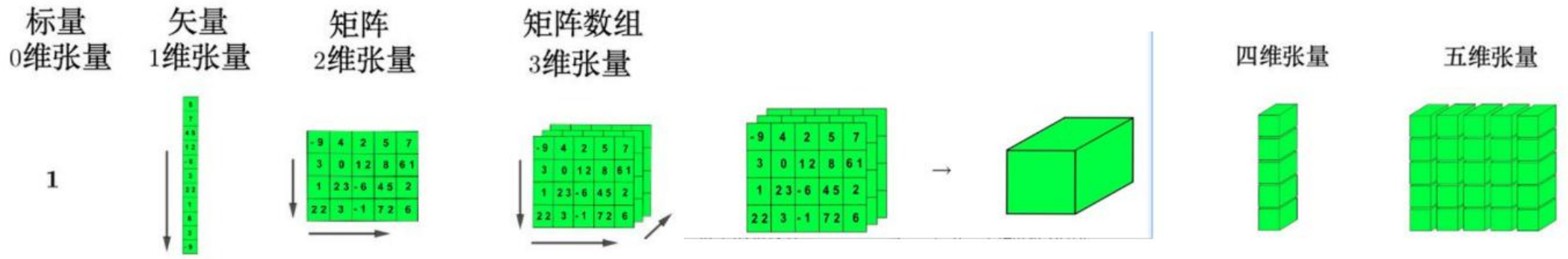
## 什么是张量

PyTorch中的张量就是元素为同一种数据类型的**多维矩阵**。在PyTorch中，张量以"类"的形式封装起来，对张量的一些运算、处理的方法被封装在类中。

PyTorch张量与NumPy数组类似，但PyTorch的张量具有GPU加速的能力（通过CUDA），这使得深度学习模型能够高效地在GPU上运行。

PyTorch提供了对张量的强大支持，可以进行高效的**数值计算**、**矩阵操作**、**自动求导**等。

张量是 PyTorch 中的**核心数据抽象**，PyTorch 支持各种张量子类型。通常地，一维张量称为向量/矢量（vector），二维张量称为矩阵（matrix）。



# 学习目标

Learning Objectives

1. 掌握张量创建方法
2. 知道线性和随机张量的创建方法
3. 知道0-1张量的创建方法
4. 知道张量元素类型的转换方法

## 张量基本创建方式

- torch.tensor 根据指定数据创建张量
- torch.Tensor 根据形状创建张量，其也可用来创建指定数据的张量
- torch.IntTensor、torch.FloatTensor、torch.DoubleTensor 创建指定类型的张量

## 创建线性和随机张量

- torch.arange() 和 torch.linspace() 创建线性张量
- torch.random.initial_seed() 和 torch.random.manual_seed() 随机种子设置
- torch.rand/randn() 创建随机浮点类型张量
- torch.randint(low, high, size=()) 创建随机整数类型张量

## 创建0、1、指定值张量

- torch.ones 和 torch.ones_like 创建全1张量
- torch.zeros 和 torch.zeros_like 创建全0张量
- torch.full 和 torch.full_like 创建全为指定值张量

## 张量元素类型转换

- data.short()/int()/long()/half()/float()/double()
- data.type(torch.short/int/long/half/float/double)

# 总结

1.**创建张量的方式**

- torch.tensor() **根据指定数据创建张量,也可以指定类型**
- torch.Tensor() **根据形状创建张量, 其也可用来创建指定数据的张量**
- torch.IntTensor()、torch.FloatTensor()... **创建指定类型的张量**

2.**创建线性和随机张量**

- torch.arrange() **和** torch.linspace() **创建线性张量**
- torch.random.initial_seed() **和** torch.random.manual_seed() **随机种子设置**
- torch.randn() **创建随机张量**

3.**创建0、1、指定值张量**

- torch.ones() **和** torch.ones_like() **创建全1张量**
- torch.zeros() **和** torch.zeros_like() **创建全0张量**
- torch.full() **和** torch.full_like() **创建全为指定值张量**

4.**张量元素类型转换**

- data.short()/int()/long()/half()/float()/double()
- data.type(torch.short/int/long/half/float/double)

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

# 学习目标

Learning Objectives

1. 掌握张量转换为Numpy数组的方法
2. 掌握Numpy数组转换为张量的方法
3. 掌握标量张量和数字转换方法

## 张量和NumPy互转

张量转换为 numpy 数组

- data_tensor.numpy() ：共享内存
- data_tensor.numpy().copy() ：不共享内存

numpy 转换为张量

- torch.from_numpy(data_numpy) ：共享内存
- torch.tensor(data_numpy) ：不共享内存

## 张量和标量互转

- torch.tensor(标量)
- tensor.item()

# 总结

1. 张量转换为 numpy 数组

- data_tensor.numpy()
- data_tensor.numpy().copy()

2. numpy 转换为张量

- torch.from_numpy(data_numpy)
- torch.tensor(data_numpy)

3. 标量张量和数字转换

- tensor.item()

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

# 学习目标

Learning Objectives

1. 掌握张量基本运算
2. 掌握张量矩阵乘法运算

## 张量基本运算

1. 张量基本运算函数

- + - * /
- add、sub、mul、div、neg等函数
- add_、sub_、mul_、div_、neg_等函数

2. 矩阵乘法运算

- @
- matmul

# 总结

1. 张量基本运算函数

- + - * /
- add、sub、mul、div、neg等函数
- add_、sub_、mul_、div_、neg_等函数

2. 矩阵乘法运算

- @
- matmul

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

# 学习目标

Learning Objectives

1. 掌握张量相关的运算函数

## 常见运算函数

PyTorch 为每个张量封装很多实用的计算函数：

- 均值
- 平方根
- 求和
- 指数计算
- 对数计算等等
- ...

# 总结

1. 张量运算函数

sum, mean, sqrt, pow, exp, log等

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

# 学习目标

Learning Objectives

1. 掌握简单行列索引的使用
2. 掌握列表索引的使用
3. 掌握范围索引的使用
4. 知道布尔索引的使用
5. 知道多维索引的使用

## 索引操作

我们在操作张量时，经常需要去获取某些元素就进行处理或者修改操作，在这里我们需要了解在torch中的索引操作

张量中根据索引获取对应位置的数据：

**格式**：张量[行,列]

**注意**：上述格式中行和列的表示形式可以是：单索引,列表,切片,布尔索引

# 总结

张量中根据索引获取对应位置的数据：

**格式**：张量[行, 列]

**注意**：行和列表示形式：单索引, 列表, 切片, 布尔索引

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

# 学习目标

Learning Objectives

1. 掌握reshape()、squeze()、unsqueeze()、transpose()、permute()、等函数使用

## reshape()函数

reshape 函数可以在保证张量数据不变的前提下改变数据的维度，将其转换成指定的形状。

## view()和contiguous()函数

view 函数也可以用于修改张量的形状，只能用于修改连续的张量。在 PyTorch 中，有些张量的底层数据在内存中的存储顺序与其在张量中的逻辑顺序不一致，view 函数无法对这样的张量进行变形处理，例如: 一个张量经过了 transpose 或者 permute 函数的处理之后，就无法使用 view 函数进行形状操作。

## squeeze()和unsqueeze()函数

squeeze 函数删除形状为 1 的维度（降维），unsqueeze 函数添加形状为1的维度（升维）。

## transpose()和permute()函数

transpose 函数可以实现交换张量形状的指定维度, 例如: 一个张量的形状为 (2, 3, 4) 可以通过 transpose 函数把 3 和 4 进行交换, 将张量的形状变为 (2, 4, 3) 。 permute 函数可以一次交换更多的维度。

# 总结

1. reshape 函数可以在保证张量数据不变的前提下改变数据的维度

2. squeeze 和 unsqueeze 函数可以用来增加或者减少维度

3. transpose 函数可以实现交换张量形状的指定维度，permute 可以一次交换更多的维度

4. view 函数也可以用于修改张量的形状，但是它要求被转换的张量内存必须连续，所以一般配合 contiguous 函数使用

# 目录

Contents


- 张量的创建
- 张量的类型转换
- 张量数值计算
- 张量运算函数
- 张量索引操作
- 张量形状操作
- 张量拼接操作

# 学习目标

Learning Objectives

1. 掌握torch.cat()使用
2. 掌握torch.stack()使用

## torch.cat()

torch.cat()函数可以将多个张量根据指定的维度拼接起来，不改变维度数。

```
import torch
data1 = torch.randint(0, 10, [1, 2, 3])
data2 = torch.randint(0, 10, [1, 2, 3])
print(data1)
print(data2)
# 1. 按0维度拼接
new_data = torch.cat([data1, data2], dim=0)
print(new_data)
print(new_data.shape)
# 2. 按1维度拼接
new_data = torch.cat([data1, data2], dim=1)
print(new_data)
print(new_data.shape)
# 3. 按2维度拼接
new_data = torch.cat([data1, data2], dim=2)
print(new_data)
print(new_data.shape)
```

**输出结果:**

```
tensor([[[7, 8, 7],
         [6, 3, 6]]])
tensor([[[3, 6, 5],
         [7, 5, 0]]])
tensor([[[7, 8, 7],
         [6, 3, 6]],

        [[3, 6, 5],
         [7, 5, 0]]])
torch.Size([2, 2, 3])
tensor([[[7, 8, 7],
         [6, 3, 6],
         [3, 6, 5],
         [7, 5, 0]]])
torch.Size([1, 4, 3])
tensor([[[7, 8, 7, 3, 6, 5],
         [6, 3, 6, 7, 5, 0]]])
torch.Size([1, 2, 6])
```

## torch.stack()

torch.stack()函数会在一个新的维度上连接一系列张量，这会增加一个新维度，并且所有输入张量的形状必须完全相同。

```
import torch
data1 = torch.randint(0, 10, [2, 3])
data2 = torch.randint(0, 10, [2, 3])
print(data1)
print(data2)
# 1. 在0维度上拼接
new_data = torch.stack([data1, data2], dim=0)
print(new_data)
print(new_data.shape)
# 2. 在1维度上拼接
new_data = torch.stack([data1, data2], dim=1)
print(new_data)
print(new_data.shape)
# 3. 在2维度上拼接
new_data = torch.stack([data1, data2], dim=2)
print(new_data)
print(new_data.shape)
```

**输出结果:**

```
tensor([[[6, 2, 8],
         [9, 8, 3]],

        [[9, 2, 8],
         [0, 7, 9]]])
torch.Size([2, 2, 3])
tensor([[[6, 2, 8],
         [9, 2, 8]],

        [[9, 8, 3],
         [0, 7, 9]]])
torch.Size([2, 2, 3])
tensor([[[6, 9],
         [2, 2],
         [8, 8]],

        [[9, 0],
         [8, 7],
         [3, 9]]])
```

# 总结

1. cat()函数可以将张量按照指定的维度拼接起来
2. stack函数可以将张量在新维度上拼接起来

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌